

Project: Code Review Desk

A spec-driven build — requirements only, two hours

This is a specification, not a tutorial. There are no prompts to copy. You will write a spec, hand it to your coding agent — Claude Code or OpenCode — and answer for everything it produces.

The brief. The Desk reviews a code change. A unified diff goes in; three reviewers — security, tests and style — read it **at the same time**, each tuned differently; their findings merge into one structured report. A critical security finding hands the conversation to a Remediation agent that proposes the fix. Nothing leaves the Desk that quotes a secret it found in the diff. Findings appear in the interface as they land, not in one lump at the end, and the whole review is a single trace in which you can name the slowest reviewer.

You have built a conversational agent before. This one is a **pipeline**: it fans out, runs in parallel, and fans back in. Everything in the fundamentals guide is used.

The rules

1. **Specification before implementation.** No source file is written until Phase 0 is committed. Your git history is the evidence and it is graded.
2. **The agent writes the code. You own it.** Anything you cannot explain, you have not finished.
3. **Requirements are numbered and testable.** "FR-5 works" means you can show it on demand.
4. **The clock is real.** Falling behind means cutting from the bottom of the cut list, not from whatever looks hardest.

The clock

Time	Phase	What exists at the end of it
0:00–0:35	Phase 0 — specify	Four spec artifacts, committed, no code
0:35–1:05	Phase 1 — intake and one reviewer	FR-1 ... FR-4
1:05–1:40	Phase 2 — fan out	FR-5 ... FR-9
1:40–1:55	Phase 3 — observe and ship	FR-10 ... FR-13
1:55–2:00	Demo	One diff reviewed live, streamed, traced

Cut list, in this order: FR-11, then FR-7, then the agent-level hooks in FR-10. Never cut FR-5 or FR-8 — the concurrency and the output guardrail are what this project is for, and what the viva asks about.

Phase 0 — the specification gate

Four documents before any implementation. Spec Kit produces this shape; plain markdown is fine as long as all four exist and say the same things.

constitution.md — the rules the build may not break. At minimum: the provider and the level it is configured at; secrets live only in `.env` and never in output; no tool raises into the runner; a review must be reproducible from the diff alone.

spec.md — what the Desk does, as behaviour. Every requirement below in your own words, plus the three things this project deliberately will not do.

plan.md — the architecture: which agents exist, which run concurrently, what each tool takes and returns, and the exact shape of every structure crossing a boundary.

tasks.md — ordered implementation tasks, each independently verifiable, each naming the requirement it serves.

Gate: all four committed before the first line of code. One commit containing both a spec and an implementation fails this phase even if the code runs.

Phase 1 — intake and one reviewer

FR-1 — A diff goes in, split by file

The Desk reads a unified diff from a path given on the command line and splits it into per-file chunks before any model sees it. The model is `gemini-2.5-flash`, configured on the agent itself. The entry point is asynchronous.

Done when: a two-file diff produces two chunks; an empty or malformed diff is reported as a message, not a traceback; nothing in the code sets a global default client.

FR-2 — Repository rules live in context

A `ReviewContext` is passed to every run and read by tools. It never appears in prompt text.

```
from dataclasses import dataclass

@dataclass
class ReviewContext:
    repo: str
    language: str
    ruleset_id: str
    strictness: str = "normal"      # "normal" or "strict"
```

Done when: a tool reads `ruleset_id` through the wrapper, the generated schema for that tool has no wrapper parameter, and grepping your prompts finds no repository name.

FR-3 — Findings come back as a list of typed objects

A reviewer returns a list, not prose.

```
from typing import Literal
from pydantic import BaseModel

class Finding(BaseModel):
    file: str
    line: int
    severity: Literal["critical", "major", "minor"]
    message: str
```

The agent's `output_type` is `list[Finding]`. Look at the generated schema before you trust it: the SDK wraps a list root in an object with a single `response` key, because strict schemas must be objects. What you get back in `final_output` is still a plain Python list.

Done when: you can iterate `final_output` and count criticals with a Python expression, and you can point at the wrapper in the generated schema and explain why it is there.

FR-4 — Reviewer instructions are built per run

The reviewer's system prompt is assembled at request time from the ruleset and the language in context, and gets terser when `strictness` is `strict`.

Done when: two contexts produce two visibly different prompts, and you can print the resolved prompt before any model call.

Phase 2 — fan out

FR-5 — Three reviewers, cloned, running concurrently

Security, tests and style reviewers are clones of one base reviewer, differing in instructions and model settings. They run **concurrently** over the same diff, not one after another.

Done when: the three reviews are launched together and awaited as a group; the wall clock for three concurrent reviews is close to the slowest single review, not the sum of all three; and you can show both numbers. A sequential version that merely works does not satisfy this requirement.

FR-6 — Merge as a tool, remediation by handoff

Two specialists, wired two different ways on purpose:

- A **Merge** specialist, exposed with `as_tool`, deduplicates overlapping findings and orders them by severity. The Desk keeps the conversation.
- A **Remediation** specialist, reached by `handoff`, takes over when a critical security finding exists, and proposes the patch directly to the user.

Done when: both paths fire on the right kind of diff, and your `spec.md` argues in two sentences why merging is a tool call while remediation is a transfer.

FR-7 — A cheaper second opinion, configured at the run level

The Desk can re-run a review on a cheaper model without touching any agent definition — the override happens on the run.

Done when: the same reviewer object produces one review on its own model and one on the override, and no agent's `model=` changed between them.

FR-8 — Nothing leaks: an output guardrail

An output guardrail inspects the finished report and refuses it if it contains anything shaped like a credential — an API key, token or password copied out of the diff. The program catches the tripwire and reports the refusal; it does not crash.

Done when: a diff containing a fake key produces a refusal rather than a report, a clean diff passes untouched, and you can point at the line that caught the exception.

FR-9 — Required tools, failing tools, and a ceiling

Three controls, all present:

- The reviewer that must consult the ruleset is configured so the model has no choice but to call it.
- The diff-reading tools hand their failures to a dedicated error handler rather than raising.
- Every review runs under a turn ceiling that raises, is caught, and is reported as a partial review.

Done when: deleting the ruleset file produces a review that still finishes with a sensible message, and you can state the ceiling you chose and the reasoning behind the number.

Phase 3 — observe and ship

FR-10 — Latency and tokens per reviewer

Run-level hooks record, for each reviewer, how long it took and how many tokens it used, and the report carries those numbers in a footer. Agent-level hooks are attached to exactly one reviewer.

Done when: the footer shows three rows with token counts read from the run context, not estimated, and you can explain what the agent-level hooks see that the run-level hooks do not.

FR-11 — Every run lands in a ledger

A custom runner appends one line per run to `ledger.jsonl`, registered once at startup. No agent definition mentions it.

```
{"ts": "2026-09-23T19:04:11Z", "request_id": "rev_8f21", "agent": "SecurityReviewer",  
  "ms": 2140, "findings": 3}
```

Done when: one review of a three-file diff produces one ledger line per run, and removing the registration is the only change needed to switch the ledger off.

FR-12 — Findings stream into the interface

A Chainlit page takes a pasted diff and shows findings as they arrive rather than after everything finishes. Session state holds the context and the last report.

Done when: text appears progressively during a review, a second diff in the same session reuses the existing context, and the handler awaits its run rather than calling the synchronous variant.

FR-13 — One review, one trace

Tracing is enabled and exported under your own key. A whole review — all three reviewers, the merge, any handoff — appears as one trace.

Done when: you can open the trace, see the three reviewers overlapping in time rather than stacked end to end, and name the slowest one.

Non-functional requirements

- **NFR-1 — Secrets.** Keys live in `.env`, gitignored. A missing key fails at startup with a sentence, not a stack trace. No secret is ever written to the ledger or the report.
- **NFR-2 — Cost.** Every agent declares its own model settings. No unbounded generation anywhere.
- **NFR-3 — Observability.** Every review is traceable and every run is in the ledger.
- **NFR-4 — Failure.** A tool that meets bad input returns a sentence the model can use. A tool that raises into the runner is a defect.
- **NFR-5 — Provenance.** `git log` shows the four Phase 0 artifacts committed before the first code commit. This is checked.

Concept coverage

Part	Forced by
0–2 Setup, keys, Gemini	FR-1, NFR-1
3 Runner , asyncio , streaming	FR-1, FR-5, FR-12
4 Model configuration	FR-1 (agent level), FR-7 (run level)
5 Tools	FR-1, FR-2
6 Model settings	FR-5, FR-9, NFR-2
7 Local context	FR-2
8 Dynamic instructions	FR-4
9 Cloning	FR-5
10 Tracing	FR-13
11 Agents as tools	FR-6 (merge)
12 Handoffs	FR-6 (remediation)
13 Advanced tool control	FR-9, NFR-4
14 Structured output	FR-3
15 Guardrails	FR-8 (output side)
16 Lifecycle hooks	FR-10
17 Run lifecycle hooks	FR-10
18 Custom runners	FR-11
19 Chainlit	FR-12
20 Practice with an agent CLI	The whole project — you are driving one

Definition of done

Requirement	How it is checked
Spec preceded code	<code>git log</code> order, artifacts first
Diff split before any model call	Two-file diff, two chunks
Context never in the prompt	Tool schema has no wrapper; grep the prompts
Findings are typed	Criticals counted with a Python expression
Reviewers run concurrently	Two wall-clock numbers, side by side
Merge is a tool, remediation is a handoff	Both fire; two-sentence justification
Cheaper re-run without edits	Same agent object, two models
Secrets never printed	Planted key produces a refusal
Tokens are measured, not guessed	Footer rows from the run context
One review is one trace	Overlapping reviewer spans, slowest named

Defending your work

1. Show the wall clock for concurrent and sequential runs. What exactly made the difference?
2. Your findings arrive as a list. What shape does the model actually emit, and why is it not a list?
3. Which parts of the three reviewers are shared with the base, and which are their own?
4. Why is merging a tool call and remediation a handoff? What would break if you swapped them?
5. The output guardrail fired. At what point in the run, and what had already been paid for?
6. Where do the token numbers in your footer come from?
7. Your ledger has more lines than you expected. Which runs produced the extra ones?
8. Which of your controls would stop a reviewer that started calling a tool in a loop?

If you finish early

- Give the remediation handoff a typed input so the Desk must state which finding triggered it.
- Add a second output guardrail that rejects any finding whose `line` is not in the diff.
- Persist reports so the Desk can answer "what changed since the last review of this repo?".
- Point it at a real pull request from your own repository and see which reviewer is useless.